INF461 · PRINCIPES ET PROCESSUS DU LOGICIEL

Le patron Builder

Exemple du cours : construction de pizzas
Variante : configuration d'ordinateurs

Étudiant : Theodore Wilfried Bitjong

Matricule : 23U2331

Dépôt GitHub :
https://github.com/inf461-bitjong/builder

Organisation GitHub :
https://github.com/inf461-bitjong

11 novembre 2026

1. Présentation du patron

Builder est un patron de création qui sépare la construction d'un objet complexe de sa représentation. Un client choisit un monteur concret et lui demande de construire le produit ; les étapes et les valeurs assemblées peuvent ainsi varier sans changer le produit final ni le code client.

Rôles du modèle générique

Rôle	Responsabilité
Produit	Objet construit et remis au client.
Builder abstrait	Définit l'accès au produit et les opérations de construction disponibles.
Builder concret	Réalise les opérations et configure le produit.
Directeur ou configurateur	Orchestre les étapes ou fournit des configurations prédéfinies.
Client	Demande la construction et utilise le produit obtenu.

> **Choix du modèle générique.** Dans l'exemple du cours, une pizza est assemblée en plusieurs étapes qui dépendent de la recette. Dans la variante, un ordinateur est configuré avec plusieurs caractéristiques indépendantes, et les méthodes chaînables du builder rendent ces configurations lisibles. Dans les deux cas, Builder isole la construction du produit de son utilisation.

Exemples présentés

- **Cours :** `Directeur` choisit un monteur de pizza et retourne la pizza terminée.
- **Variante :** `Configurator` prépare des profils de jeu et de bureau au moyen d'un builder fluide.

2. Exemple vu en cours : les pizzas

Le produit `Pizza` contient une pâte, une sauce et une garniture. La classe abstraite `MonteurPizza` définit les étapes de construction et les exécute dans `getPizza()`. Les monteurs concrets fournissent les valeurs propres aux pizzas Reine et Piquante. `Directeur` sélectionne le monteur correspondant.

Modélisation

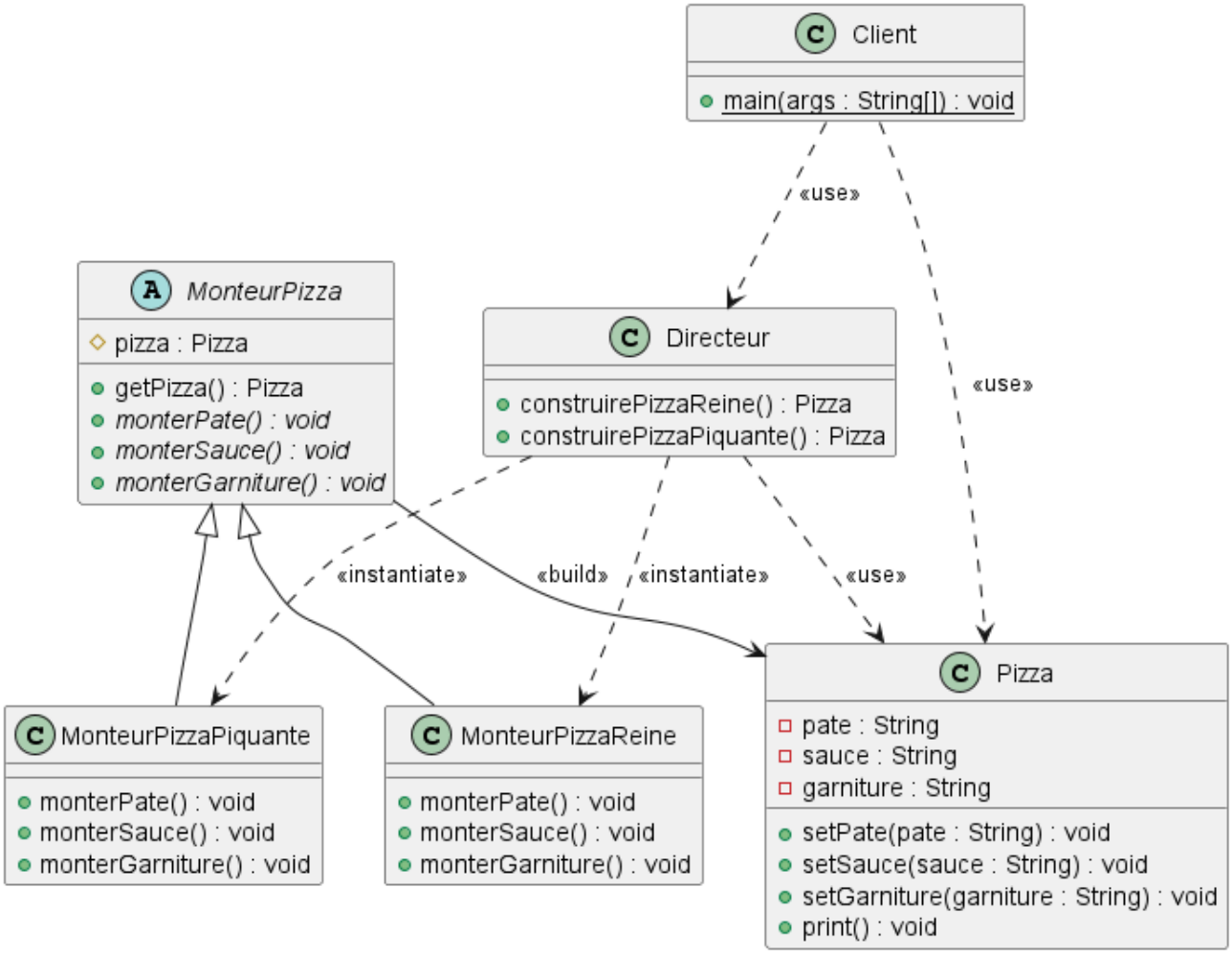


Figure 1 — Diagramme de classes de l'exemple du cours.

> **Analyse.** Le modèle classique avec un directeur, un builder abstrait, des builders concrets et un produit convient ici parce que les étapes (pâte, sauce, garniture) sont communes, tandis que leur résultat change selon la recette.

3. Implémentation de l'exemple du cours

Client.java

```
package cours;

public class Client {
    public static void main(String[] args) {
        Directeur directeur = new Directeur();

        System.out.println(".....Pizza Reine Fabriquée.....");
        directeur.construirePizzaReine().print();

        System.out.println("\n...Pizza Piquante Fabriquée...");
        directeur.construirePizzaPiquante().print();
    }
}
```

Directeur.java

```
package cours;

import cours.abstractions.MonteurPizza;
import cours.builders.MonteurPizzaPiquante;
import cours.builders.MonteurPizzaReine;
import cours.modeles.Pizza;

public class Directeur {
    public Pizza construirePizzaReine() {
        MonteurPizza pizzaReine = new MonteurPizzaReine();
        return pizzaReine.getPizza();
    }

    public Pizza construirePizzaPiquante() {
        MonteurPizza pizzaPiquante = new MonteurPizzaPiquante();
        return pizzaPiquante.getPizza();
    }
}
```

3. Implémentation de l'exemple du cours

MonteurPizza.java

```
package cours.abstractions;

import cours.modeles.Pizza;

public abstract class MonteurPizza {
    protected Pizza pizza = new Pizza();

    public Pizza getPizza() {
        monterPate();
        monterSauce();
        monterGarniture();
        return pizza;
    }

    public abstract void monterPate();

    public abstract void monterSauce();

    public abstract void monterGarniture();
}
```

MonteurPizzaReine.java

```
package cours.builders;

import cours.abstractions.MonteurPizza;

public class MonteurPizzaReine extends MonteurPizza {
    @Override
    public void monterPate() {
        pizza.setPate("croisée");
    }

    @Override
    public void monterSauce() {
        pizza.setSauce("douce");
    }

    @Override
    public void monterGarniture() {
        pizza.setGarniture("jambon+champignon");
    }
}
```

Le patron Builder · Code source du cours

3. Implémentation de l'exemple du cours

MonteurPizzaPiquante.java

```
package cours.builders;

import cours.abstractions.MonteurPizza;

public class MonteurPizzaPiquante extends MonteurPizza {
    @Override
    public void monterPate() {
        pizza.setPate("feuilletée");
    }

    @Override
    public void monterSauce() {
        pizza.setSauce("piquante");
    }

    @Override
    public void monterGarniture() {
        pizza.setGarniture("pepperoni+salami");
    }
}
```

Pizza.java

```
package cours.modeles;

public class Pizza {
    private String pate = "";
    private String sauce = "";
    private String garniture = "";

    public void setPate(String pate) {
        this.pate = pate;
    }

    public void setSauce(String sauce) {
        this.sauce = sauce;
    }

    public void setGarniture(String garniture) {
        this.garniture = garniture;
    }

    public void print() {
        System.out.println("Pate = " + this.pate + "\nSauce = " + this.sauce + "\nGarniture = " + this.garniture);
    }
}
```

4. Exemple modifié : configuration d'ordinateurs

La variante remplace la recette de pizza par la configuration d'un ordinateur. `Computer` est le produit ; `ComputerBuilder` décrit les opérations de configuration ; `CustomComputerBuilder` applique les valeurs au produit et retourne le builder pour permettre l'enchaînement. `Configurator` propose deux configurations prédéfinies.

Modélisation

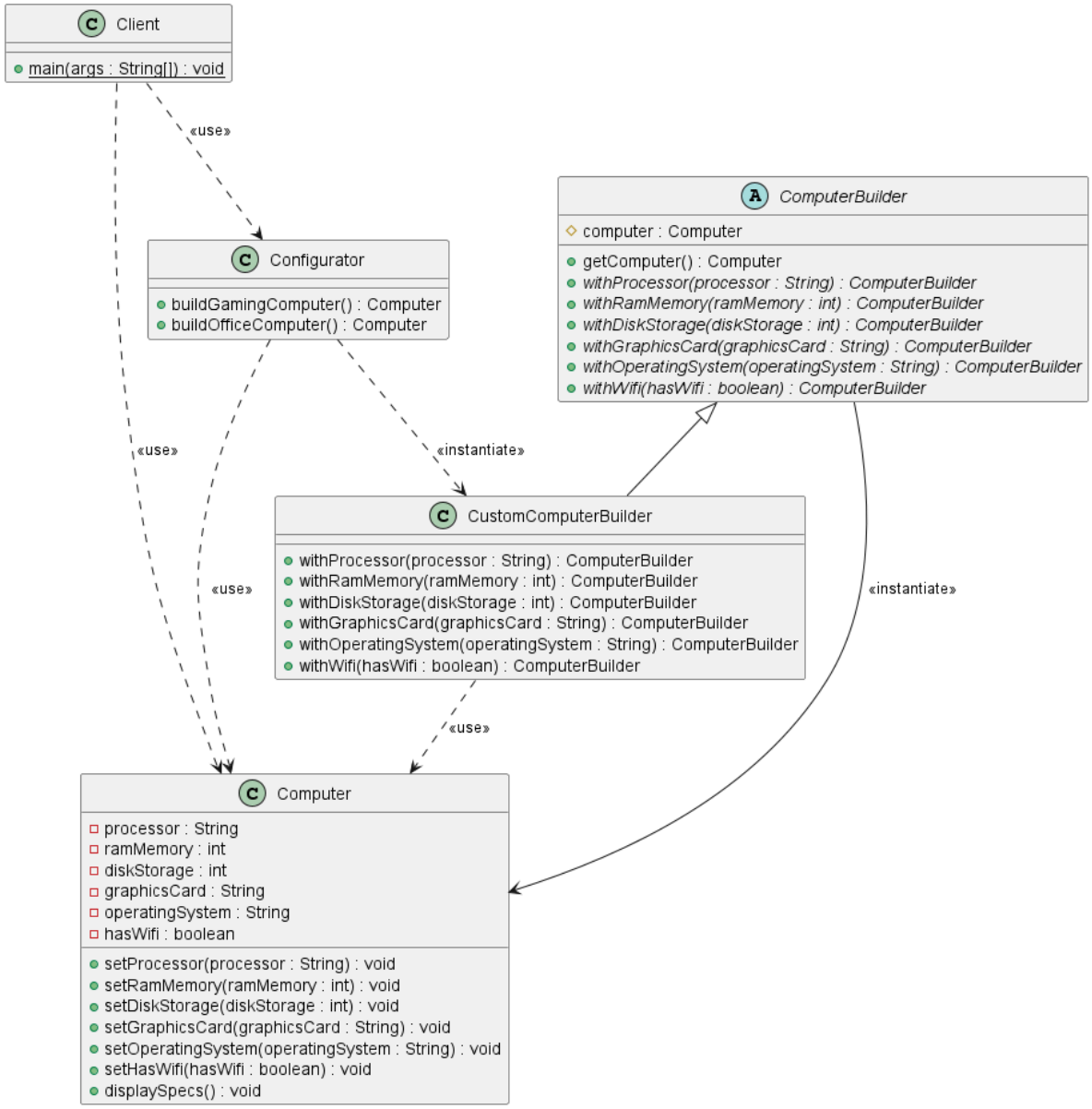


Figure 2 — Diagramme de classes de la variante pour les ordinateurs.

> **Analyse.** Le Builder fluide convient à un produit dont les paramètres sont nombreux et configurables séparément. Chaque appel `with...` modifie une caractéristique et retourne le builder ; les séquences de configuration restent lisibles et le même produit peut représenter plusieurs profils.

5. Implémentation de la variante

Client.java

```
package variante;

public class Client {
    public static void main(String[] args) {
        Configurator configurator = new Configurator();

        System.out.println("=== Configurator Gamming Computer ===");
        configurator.buildGamingComputer().displaySpecs();

        System.out.println("\n=== Configurator Office Computer ===");
        configurator.buildOfficeComputer().displaySpecs();
    }
}
```

Configurator.java

```
package variante;

import variante.builders.CustomComputerBuilder;
import variante.modeles.Computer;

public class Configurator {
    public Computer buildGamingComputer() {
        return new CustomComputerBuilder()
                .withProcessor("Intel Core i9-14900K")
                .withRamMemory(64)
                .withDiskStorage(2000)
                .withGraphicsCard("NVIDIA RTX 4090")
                .withOperatingSystem("Windows 11 Pro")
                .withWifi(true)
                .getComputer();
    }

    public Computer buildOfficeComputer() {
        return new CustomComputerBuilder()
                .withProcessor("Intel Core i5-13400")
                .withRamMemory(16)
                .withDiskStorage(512)
                .withGraphicsCard("Intel UHD Graphics")
                .withOperatingSystem("Ubuntu 24.04 LTS")
                .withWifi(false)
                .getComputer();
    }
}
```

5. Implémentation de la variante

ComputerBuilder.java

```
package variante.abstractions;

import variante.modeles.Computer;

public abstract class ComputerBuilder {
    protected Computer computer = new Computer();

    public Computer getComputer() {
        return computer;
    }

    public abstract ComputerBuilder withProcessor(String processor);

    public abstract ComputerBuilder withRamMemory(int ramMemory);

    public abstract ComputerBuilder withDiskStorage(int diskStorage);

    public abstract ComputerBuilder withGraphicsCard(String graphicsCard);

    public abstract ComputerBuilder withOperatingSystem(String operatingSystem);

    public abstract ComputerBuilder withWifi(boolean hasWifi);
}
```

CustomComputerBuilder.java

```
package variante.builders;

import variante.abstractions.ComputerBuilder;

public class CustomComputerBuilder extends ComputerBuilder {
    @Override
    public ComputerBuilder withProcessor(String processor) {
        computer.setProcessor(processor);
        return this;
    }

    @Override
    public ComputerBuilder withRamMemory(int ramMemory) {
        computer.setRamMemory(ramMemory);
        return this;
    }

    @Override
    public ComputerBuilder withDiskStorage(int diskStorage) {
        computer.setDiskStorage(diskStorage);
        return this;
    }

    @Override
    public ComputerBuilder withGraphicsCard(String graphicsCard) {
        computer.setGraphicsCard(graphicsCard);
        return this;
    }

    @Override
    public ComputerBuilder withOperatingSystem(String operatingSystem) {
        computer.setOperatingSystem(operatingSystem);
        return this;
    }

    @Override
    public ComputerBuilder withWifi(boolean hasWifi) {
        computer.setHasWifi(hasWifi);
        return this;
    }
}
```

Le patron Builder · Code source de la variante

5. Implémentation de la variante

Computer.java

```
package variante.modeles;

public class Computer {
    private String processor;
    private int ramMemory;
    private int diskStorage;
    private String graphicsCard;
    private String operatingSystem;
    private boolean hasWifi;

    public void setProcessor(String processor) {
        this.processor = processor;
    }

    public void setRamMemory(int ramMemory) {
        this.ramMemory = ramMemory;
    }

    public void setDiskStorage(int diskStorage) {
        this.diskStorage = diskStorage;
    }

    public void setGraphicsCard(String graphicsCard) {
        this.graphicsCard = graphicsCard;
    }

    public void setOperatingSystem(String operatingSystem) {
        this.operatingSystem = operatingSystem;
    }

    public void setHasWifi(boolean hasWifi) {
        this.hasWifi = hasWifi;
    }

    public void displaySpecs() {
        System.out.println("Computer Specifications:");
        System.out.println(" - Processor: " + processor);
        System.out.println(" - RAM: " + ramMemory + " GB");
        System.out.println(" - Storage: " + diskStorage + " GB");
        System.out.println(" - Graphics Card: " + graphicsCard);
        System.out.println(" - Operating System: " + operatingSystem);
        System.out.println(" - Wi-Fi: " + (hasWifi ? "Yes" : "No"));
    }
}
```

> **Exécution.** Le client demande successivement un ordinateur de jeu et un ordinateur de bureau. Dans chaque cas, le configurateur chaîne les options du builder puis récupère le `Computer` final avec `getComputer()`.

6. Dépôt et conclusion

Le dépôt contient les sources Java des deux exemples ainsi que leurs diagrammes de classes. Le code du cours et la variante sont organisés séparément sous `src/cours` et `src/variante`.

Organisation GitHub : https://github.com/inf461-bitjong

Dépôt Builder : https://github.com/inf461-bitjong/builder

> **Conclusion.** L'exemple du cours montre comment déléguer des étapes de construction communes à des monteurs spécialisés. La variante conserve la séparation entre le produit et sa création, mais emploie une API fluide adaptée au réglage de nombreuses options. Ces deux mises en œuvre illustrent comment Builder rend la construction explicite tout en laissant les détails au builder concret.